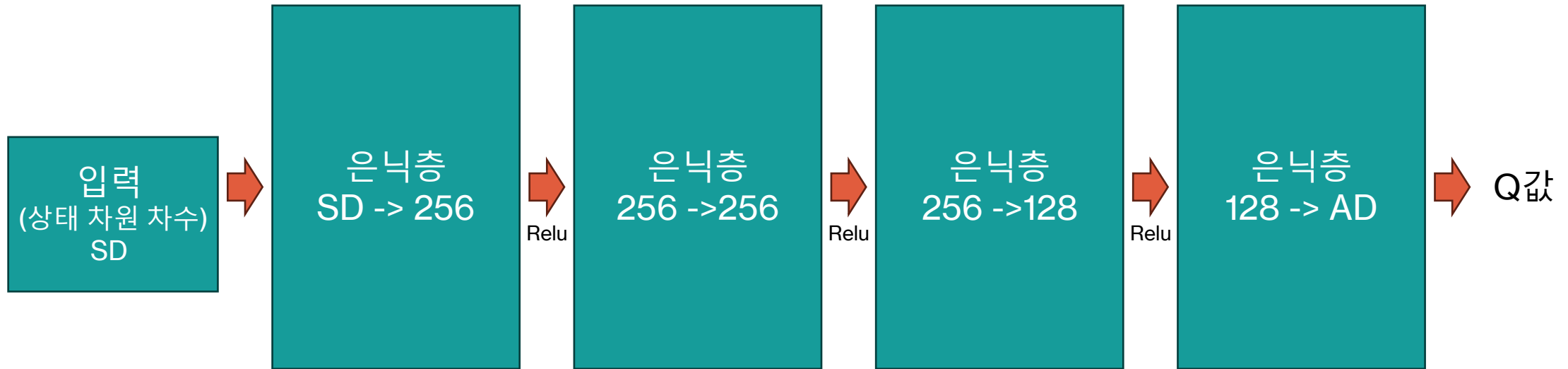


# 강화학습

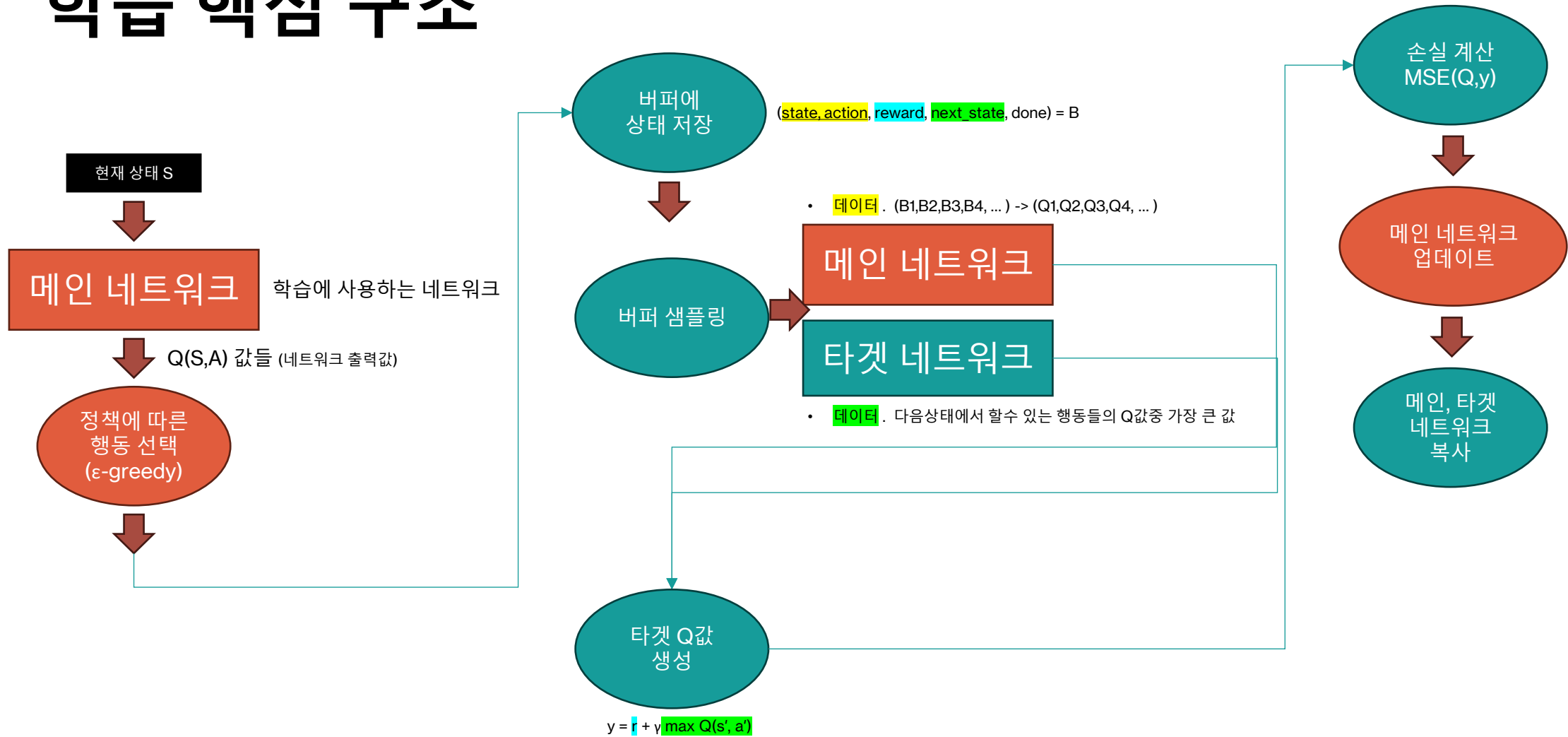
2019E7340 이준호

# 앞으로 사용한 모델 구조.



해당 구조는 장애물 회피 / 슈팅 게임 모두 공통적으로 사용됩니다

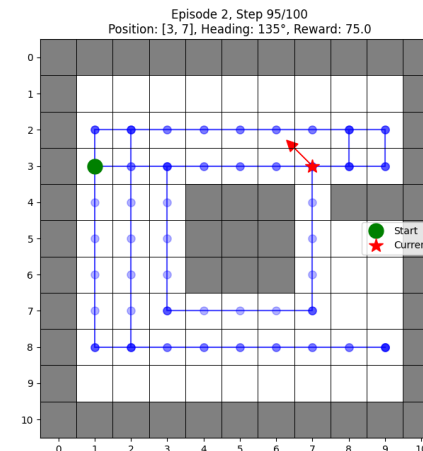
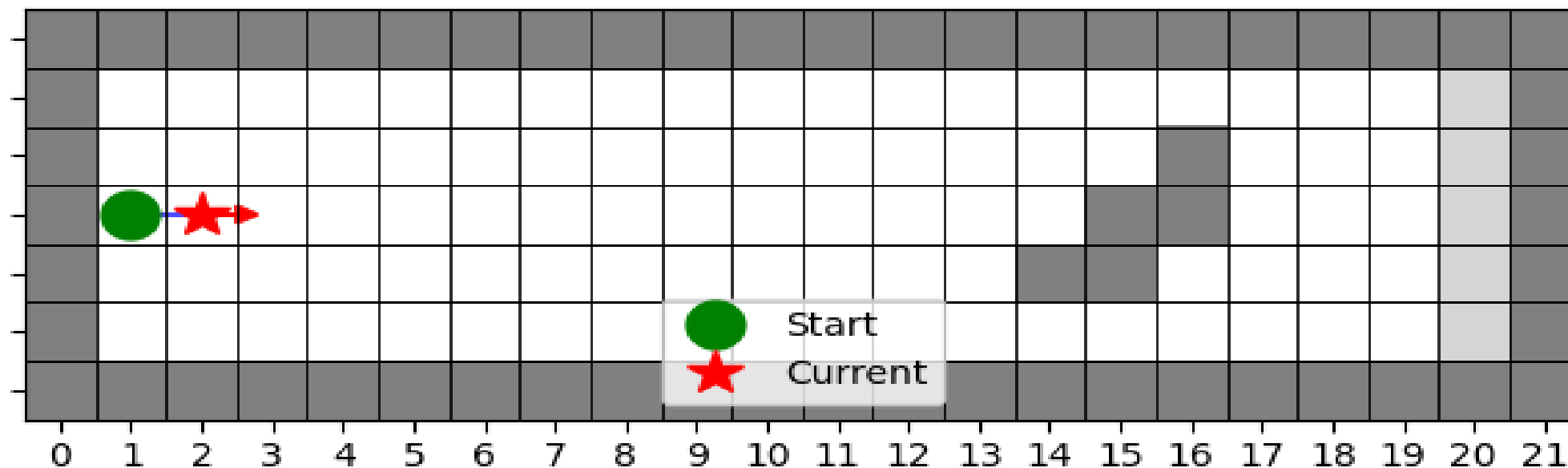
# 학습 핵심 구조



**충돌없이 오래 생존**

# 목적 소개

Episode 2000, Step 1/21  
Position: [3, 2], Heading: 0°, Reward: -1.0

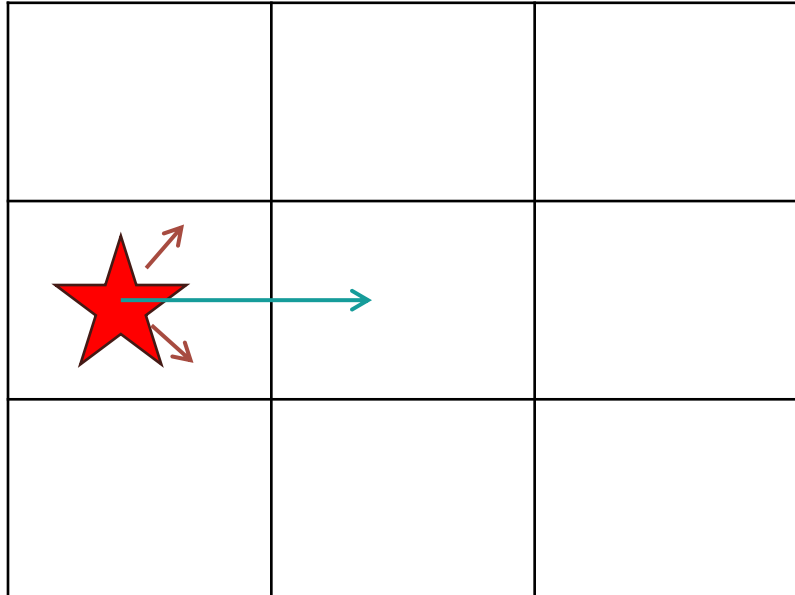
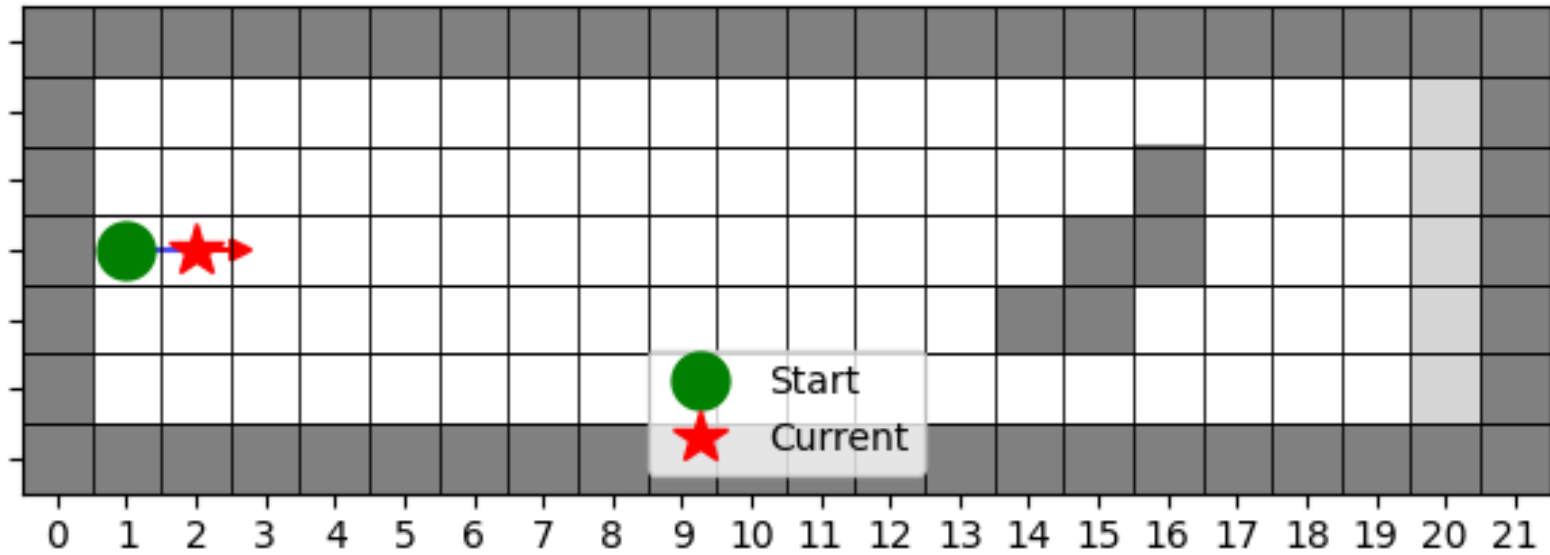


특정 맵에서 학습된 에이전트가,  
다양한 맵 구성에서도 충돌 없이 생존하며 전진할 수 있는 일반화된 정책학습

**충돌없이 오래 생존**

1차.

Episode 2000, Step 1/21  
Position: [3, 2], Heading: 0°, Reward: -1.0



상태값(입력)

x, y 위치 /

heading(방향) /

초음파 센서 거리 (-45°, 0°, +45°)

가능 행동(출력)

0: 우회전 (+45도)

1: 전진

2: 좌회전 (-45도)

## Positive Reward (보상)

**+1:** 정상 이동 성공 시

**+0.1 × 거리 차:** 목표에 가까워짐

**+0.1 × front\_distance:** 전방 장애물 멀수록  
보상

**+30:** 목표 지점 도달 성공 시



# Negative Reward (패널티)

-0.1: 기본 이동 비용

-2: 목표에서 멀어짐

-2: 전방 장애물 너무 가까움

-1: 100 스텝 초과

종료 조건:

- 벽 충돌
- 같은 위치 3회 이상 방문
- 목표에서 멀어짐 3회 누적

# 결론 및 한계.

## • 주요 결과

- 기본 맵에서는 일정 수준 이상의 생존률과 전진률 확보.
- 장애물 회피와 방향 전환의 정책이 학습됨.
- 슈팅 게임에서도 명중률 향상을 보임.

## • 한계 및 문제점

### • 다른 맵에서는 성능 저하

→ 훈련된 정책이 새로운 환경에 일반화되지 않음

### • 입력에 $(x, y)$ 위치 포함

→ 절대 위치 기반 학습으로 환경 변화에 취약

→ 상대적인 거리나 방향 기반 상태 정의가 필요

## 개선 방향

### • 상태 표현 개선

→  $x, y$  위치 제외하고 센서 기반 상대 정보만 사용

### • 데이터 다양화 및 랜덤 맵 적용

→ 일반화 성능 향상 목표

### • Curiosity 등 exploration 강화 기법 도입 고려

**충돌없이 오래 생존**

2차.

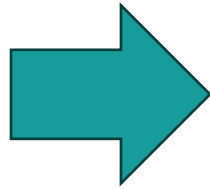
# 입력차원 축소.

상태값(입력)

x, y 위치 / 2

heading(방향) / 8

초음파 센서 거리  $(-45^\circ, 0^\circ, +45^\circ)$  3



상태값(입력)

초음파 센서 거리  $(-90^\circ, -45^\circ, 0^\circ, +45^\circ, +90^\circ)$

## Positive Reward (보상)

**+1:** 정상 이동 성공 시

~~+0.1 × 거리 차:~~ 목표에 가까워짐

~~+0.1 × front\_distance:~~ 전방 장애물 멀수록  
보상

**+30:** 목표 지점 도달 성공 시

# Negative Reward (패널티)

~~-0.1: 기본 이동 비용~~

~~-2: 목표에서 멀어짐~~

-2: 전방 장애물 너무 가까움

-1: 100 스텝 초과

종료 조건:

- 벽 충돌
- 같은 위치 3회 이상 방문
- ~~목표에서 멀어짐 3회 누적~~

## 방법별, 목표지점 도달 수(학습 도중)

| 설정                                | 목표 도달 수 | 의미 해석                           |
|-----------------------------------|---------|---------------------------------|
| 입력: (x,y,heading +<br>센서3개) + DQN | 45/3000 | 위치/방향 정보 있으니까<br>꽤 잘 감.         |
| 입력: (센서5개) + DQN                  | 23/3000 | 크게 줄어들었음.                       |
| 입력: (센서5개) +<br>SARSA             | 63/3000 | 센서만 썼지만, SARSA가<br>더 안정적으로 학습됨. |

# SARSA vs DQN

| 구분                      | SARSA                      | DQN (Q-LEARNING)                       |
|-------------------------|----------------------------|----------------------------------------|
| 업데이트 기준                 | 실제로 선택한 행동의 Q값 (on-policy) | 최적 행동(max Q) 가정하고 업데이트 (off-policy)    |
| 학습 방식                   | 신중하고 보수적 (실제 실행 기준)        | 적극적이고 공격적 (최적 가정)                      |
| 수식 (target)             | $r + \gamma * Q(s', a')$   | $r + \gamma * \max(Q(s', a'))$         |
| 학습 성향                   | 안전하고 안정적이나, 약간 느림          | 빠른 최적화 가능하지만, 불안정하거나 risky할 수 있음       |
| 탐험(exploration)과 학습의 관계 | 행동 정책과 학습 정책이 동일 (탐험 반영)   | 행동 정책과 학습 정책이 다를 수 있음 (탐험 무시하고 최선만 가정) |
| 대표 사용 환경                | 불확실성 크고 위험한 환경             | 명확한 최적 경로가 있는 환경                       |



| 구분          | SARSA                    | DQN (Q-Learning)               |
|-------------|--------------------------|--------------------------------|
| 수식 (target) | $r + \gamma * Q(s', a')$ | $r + \gamma * \max(Q(s', a'))$ |

$r$ : 직접 얻은 보상 (즉시 얻는)

$\gamma * Q(s', a')$ : 기대보상: 현재 행동 이후 미래에 받을 보상들의 합 (감가율 적용해서)

$S$  = 현재 상태

$A$  =  $S$ 에서 할 수 있는 액션들( 전진 (1) 회전 (2) )

$S'$  =  $S$  에서  $A$ 를 할 경우 의 상태 (여기서  $A$ 는 E-greedy 정책을 사용)

$A'$  =  $S'$ 에서 할 수 있는 액션들 (DQN의 경우  $\max(Q(s', a'))$  를 사용 즉 3가지 행동중 최대값을 사용

SARSA\_       $s \rightarrow (a \text{ 골라서}) \rightarrow s' \rightarrow (s' \text{에서 epsilon-greedy로 하나 골라서 사용})$

DQN\_       $s \rightarrow (a \text{ 골라서}) \rightarrow s' \rightarrow (s' \text{에서 가능한 모든 행동 중 Q값 가장 큰 걸 선택})$

# Q값 업데이트 방식.

Q-learning 을 SARSA 와 비교하면 상태(s), 행동(a), 보상(r), 다음 상태(s') 까지만 사용하는 SARS 라고 말할 수 있겠습니다. 따라서 Q-learning 과 SARSA 의 차이는 다음 행동(a') 이 학습에 필요한지 여부가 됩니다.

$$Q - learning : Q(s, a) = Q(s, a) + \alpha(R + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

$$SARSA : Q(s, a) = Q(s, a) + \alpha(R + \gamma Q(s', a') - Q(s, a))$$

# 차이는?

| 항목           | Q-LEARNING (DQN)     | SARSA                          |
|--------------|----------------------|--------------------------------|
| 학습에 필요한 것    | (s, a, r, s')까지만     | (s, a, r, s', a')까지 필요         |
| s' 도착 후?     | 모든 a' 중 최적(max Q) 찾음 | epsilon-greedy로 실제 다음 행동 a' 선택 |
| 다음 행동 a' 필요? | 필요 없음                | 필수                             |

# 결론.

DQN은 최적 행동을 가정하여 학습한다.

입력에서 위치(x, y) 및 방향(heading) 정보가 제거되면서, 최적 경로를 판단할 수 있는 근거가 약해졌다.

거리 기반 보상 항목도 삭제되어, 목표와의 상대적 거리 정보를 이용한 방향성 학습이 불가능해졌다.

결과적으로 최적 경로 가정이 현실과 괴리되어, 목표 도달 성능이 저하되었다.

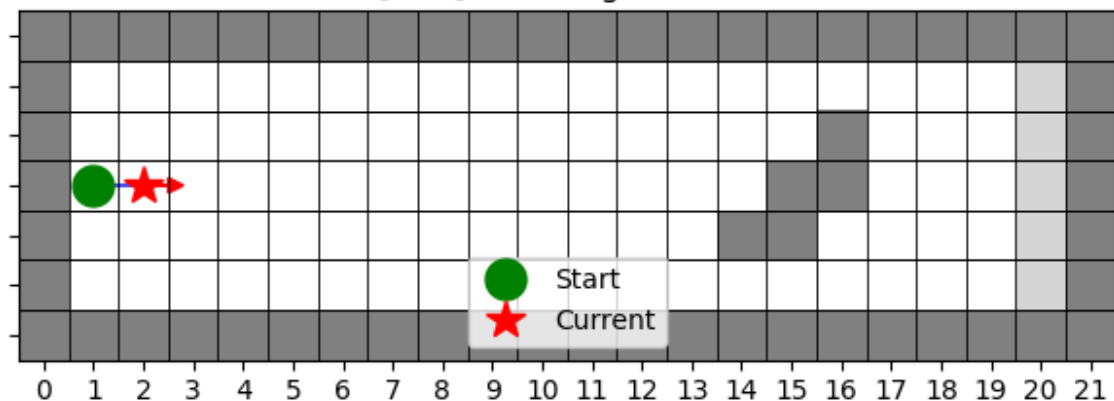
이는 인간이 현재 위치와 목적지 정보를 알지 못할 때 최적 경로를 설정하기 어려운 것과 유사한 현상이다.

1차 보단 낫지만, 다른 환경에서의 성능도 저하.

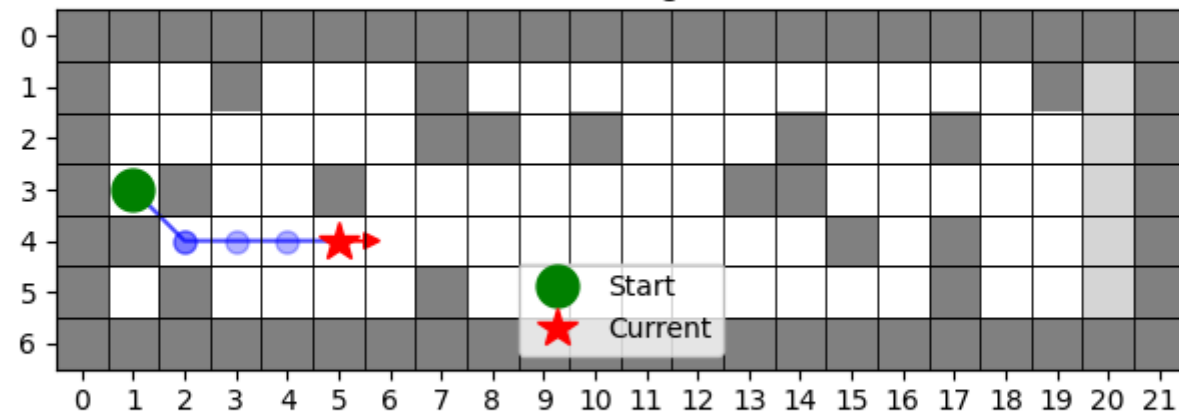
# 3차. 기존 고정 환경 학습

## -> 랜덤 환경, 보상 정책 수정

Episode 2000, Step 1/21  
Position: [3, 2], Heading: 0°, Reward: -1.0



Episode 1, Step 6/23  
Position: [4, 5], Heading: 0°, Reward: 2.0



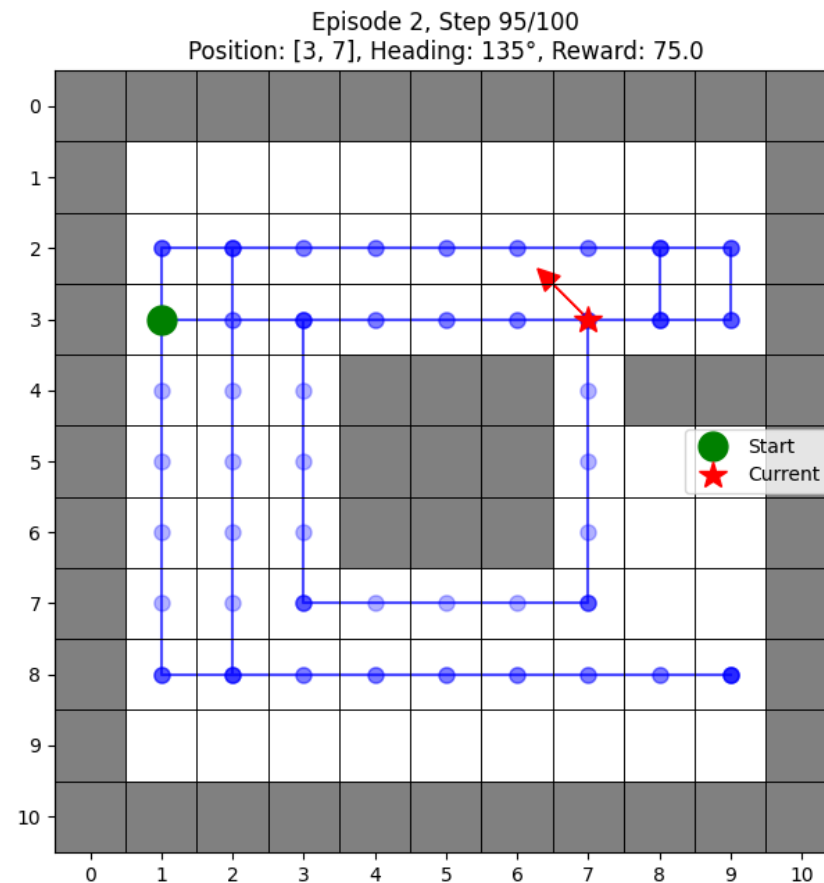
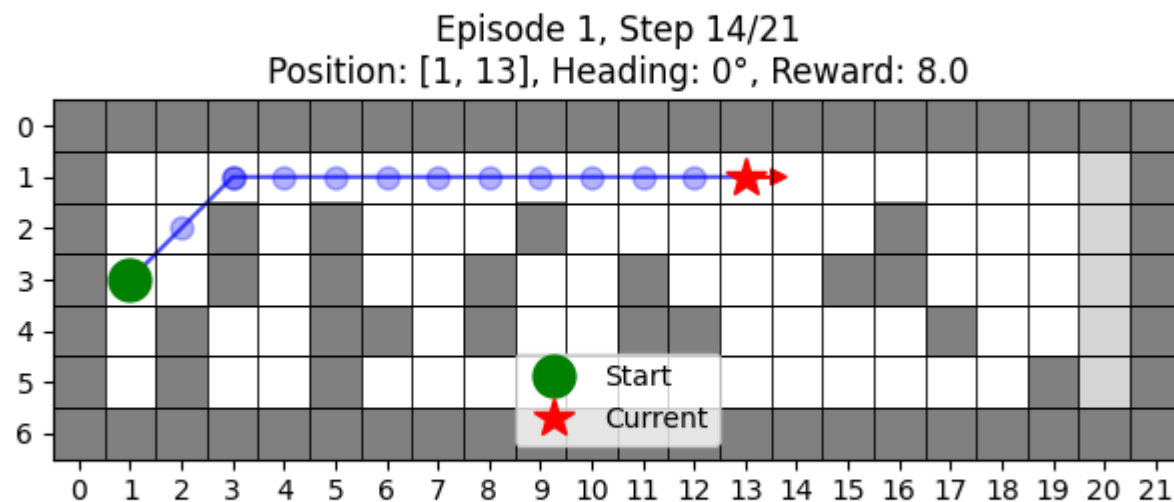
# 이전 보상과의 차이.

| 항목         | 이전 방식                     | 바꾼 방식                 |
|------------|---------------------------|-----------------------|
| 기본 보상      | -0.1 시작 → 이동 성공 시 + 1     | 이동 성공 시 + 1, 실패 시 - 1 |
| 충돌 시       | -0.1                      | -10                   |
| 반복 위치 도달 시 | 최근 5개 중 3회 이상 → 종료 (-0.1) | 최근 8개 중 4회 이상 → 종료 -5 |
| 정면 장애물 근접  | -2                        | (-2)                  |
| 목표 도달 시    | + 30                      | + 50 (더 큰 보상)         |
| 최대 스텝 초과 시 | -1                        | -10                   |
| 회전 시       | -0.1                      | -1                    |

# 바꾼 보상 정리.

| 조건                    | 보상 값     | 설명                        |
|-----------------------|----------|---------------------------|
| ✅ 정상 전진 성공            | +1       | 이동이 가능할 경우 (벽이나 장애물 아님)   |
| ❌ 장애물/벽 충돌            | -10      | 충돌 발생 시 (종료)              |
| ↺ 회전만 했을 경우           | -1       | +45도 또는 -45도 회전 시 기본 감점   |
| ⊖ 반복 위치 (비효율적 루프)     | -5       | 최근 위치에 4번 이상 등장 시 (종료)    |
| 🚫 정면 장애물 거리 1 이하      | -2 추가 감점 | 센서 기준 정면이 막혀있을 경우 (위험 인식) |
| 🎯 목표 도달 성공            | +50      | 에이전트가 목표 영역 내 도착          |
| 🕒 100스텝 초과 (시간 초과 종료) | -10      | 목표 도달 못하고 제한 시간 초과        |

# 이전에 실패했던 환경에서도 잘 동작.

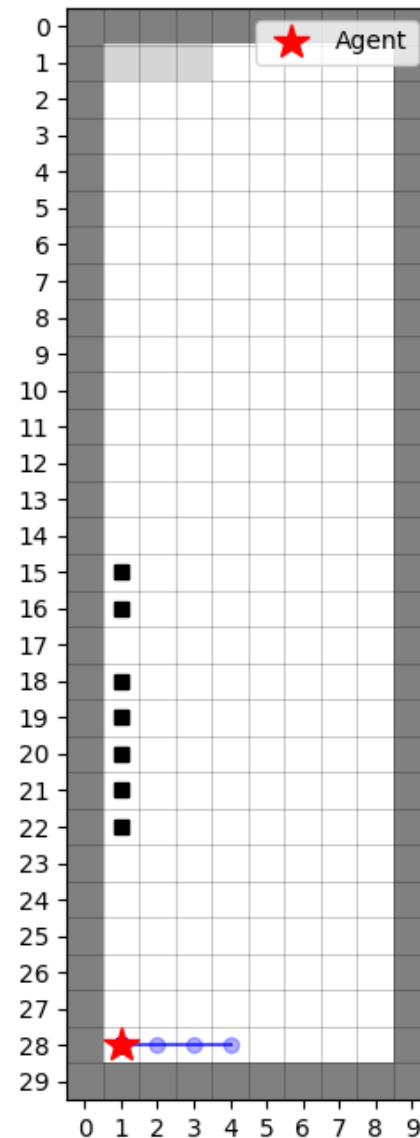




# 슈팅 게임

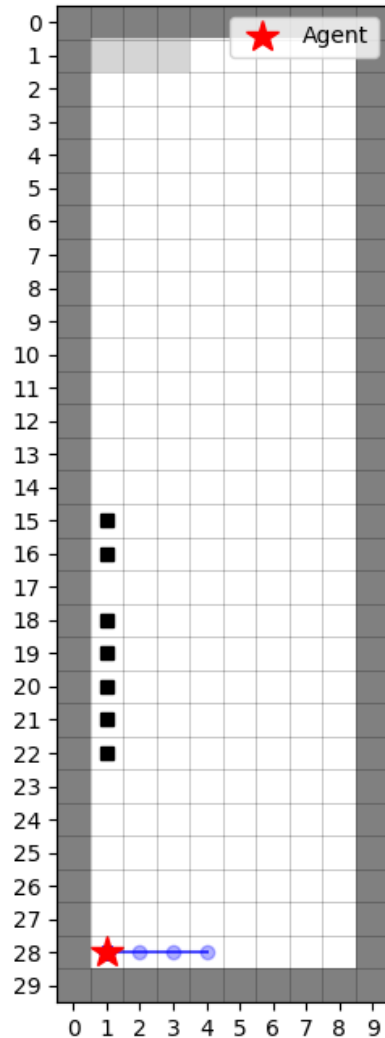
목표물을 정확히 조준하고, 불필요한  
발사를 최소화하여  
보상을 최대화하는 슈팅 정책을 학습

Episode 1, Step 16/100  
Agent Pos: (28,1), Bullets: 7, Reward: -16.0



# 슈팅게임.

Episode 1, Step 16/100  
Agent Pos: (28,1), Bullets: 7, Reward: -16.0



## 입력

### 상태(State):

•dx: 에이전트와 목표물 중심 사이의 가로 거리

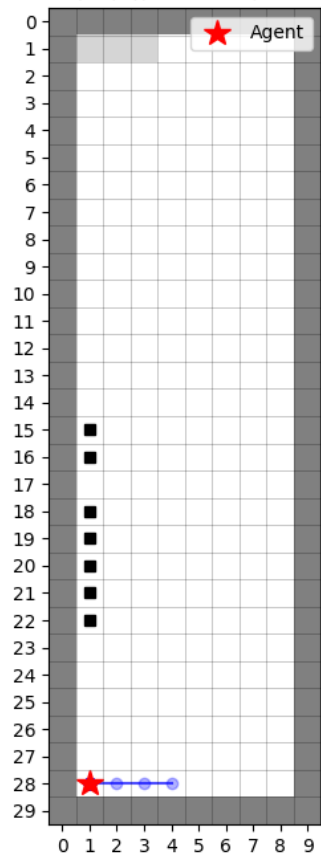
| 에이전트 위치<br>(agent_x) | 목표 중심 위치<br>(goal_center_x) | dx 값 | 의미          |
|----------------------|-----------------------------|------|-------------|
| 4                    | 4                           | 0    | 정확히 정렬됨     |
| 3                    | 4                           | 1    | 목표가 오른쪽에 1칸 |
| 6                    | 4                           | -2   | 목표가 왼쪽에 2칸  |
| 4                    | 7                           | 3    | 목표가 오른쪽에 3칸 |

## 출력

| 행동 | 설명       |
|----|----------|
| 0  | 왼쪽으로 이동  |
| 1  | 오른쪽으로 이동 |
| 2  | 제자리 정지   |
| 3  | 총알 발사    |

# 환경 구성. (Environment)

Episode 1, Step 16/100  
Agent Pos: (28,1), Bullets: 7, Reward: -16.0



## 격자 환경

- 크기: 30 (세로) x 10 (가로)
- 벽: 상하좌우 가장자리에 벽 (grid == 1)
- 에이전트 시작 위치: [28, 4] (하단 중앙)

## 🎯 목표물(Goal)

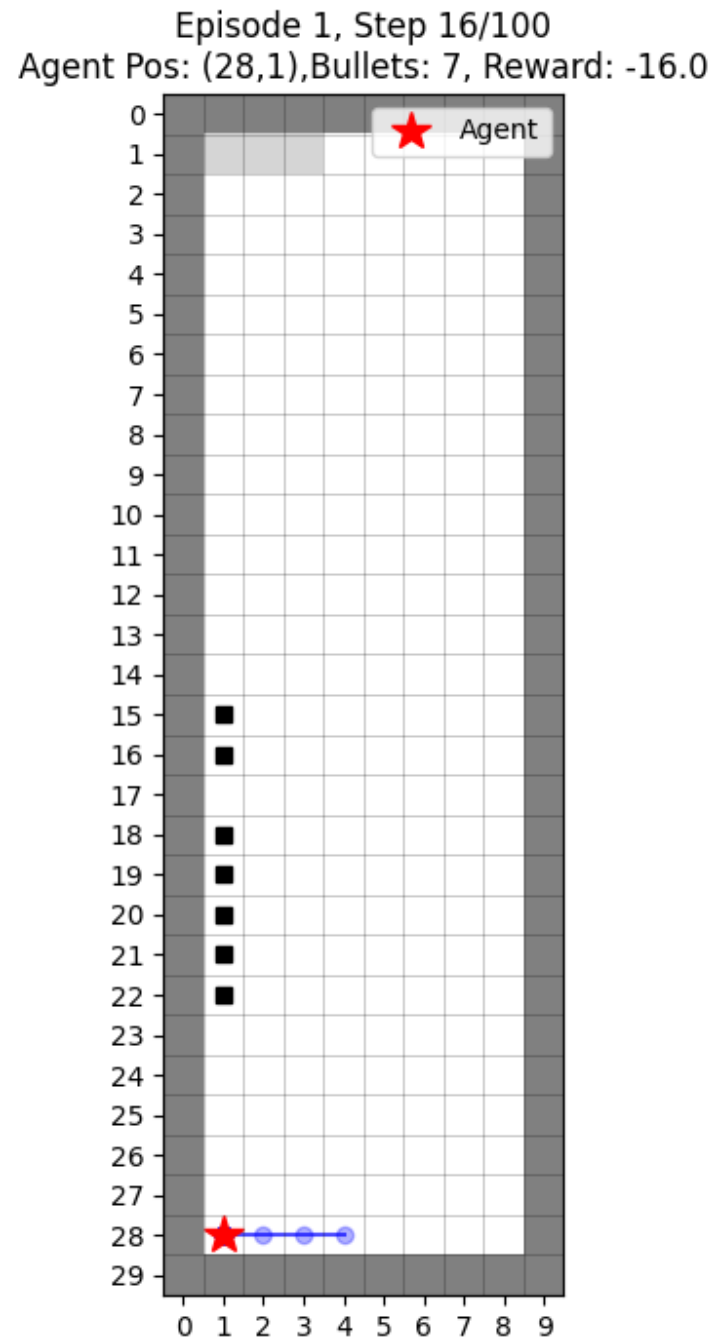
- 위치 범위: [[1, 3], [1, 5]] (작은 직사각형)
- 매 스텝마다 무작위 좌/우 이동

## 💣 총알 (Bullet)

- 발사 위치: 현재 에이전트 위치
- 매 스텝마다 위로(x -= 1) 1칸씩 전진
- 충돌 감지: 총알이 목표 사각형 안에 들어가면 hit

### 3. 보상 정책 (Reward Policy)

| 상황                   | 보상  |
|----------------------|-----|
| 총알이 목표물에 명중          | +30 |
| 총알이 목표물 없이 화면 밖으로 나감 | -4  |
| 일반 스텝 진행             | -1  |



## 4. 종료 조건 (Termination Conditions)

| 조건                                        | 설명                     |
|-------------------------------------------|------------------------|
| <code>self.hit_count &gt; 30</code>       | 목표에 30회 명중 시 성공 종료     |
| <code>self.current_steps &gt;= 100</code> | 최대 스텝 수 도달 시 종료        |
| (조기 종료) 최근 평균 명중 수 $\geq 20$              | 학습 안정 시 early stopping |

# 문제 발생 배경

- 총알 발사 후 약 **28스텝**이 지나야 목표 지점에 도달 가능
- 목표는 **랜덤하게 좌우 이동**
- 따라서 **도달 시점의 위치 예측은 매우 불확실**  
→ 에이전트 입장에서 **학습보단 운에 의존**

## 해결 방안

### 1. 목표 위치 정규화 & 입력 포함

- 목표의 X좌표를 **0.0~1.0** 사이로 정규화
  - 가장 왼쪽 → 0.0
  - 중앙 → 0.5
  - 가장 오른쪽 → 1.0
- 에이전트가 **상대적 위치 파악 가능**

### 2. 총알 개수도 정규화하여 입력

- 불필요한 발사 억제를 위해 총알 개수도 입력에 포함
- 예) 0.0(총알 없음) ~ 1.0(최대 개수 발사 중)

### 3. 최종 입력 데이터 구성

(dx, 타겟 위치 정규값, 총알 개수 정규값)

### 4. 총알 속도 2배 증가

- step당 **2칸 이동**으로 개선
- 명중 가능성 향상 → 보상 피드백 시간 단축 → 학습 안정성 증가

# 결론

## 학습 성과

- 랜덤 환경과 목표 이동에도 불구하고 안정적인 명중 가능
- 개선된 입력 구성 및 보상 정책 → 빠른 수렴, 일반화 성능 향상
- 이전 실패 환경에서도 성공적 수행

## 주요 개선 요인

- 정규화된 상대 위치 및 총알 개수 입력
- 총알 속도 2배 증가로 피드백 시간 단축
- 보상 구조 수정 → 의미 있는 행동 유도